

Woxsen University

School of Technology



B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Campus Placement and Recruitment Drive Management System

Course Code: 25TU03MJM0

Course Name: Database Management Systems

Semester: 3

Academic Year: 2025 – 29

Submitted by:

Name: Sahasra Devunuri , Pittala Siri Raaga , Riddhi Agarwal, Mohak Gupta

Register Number: 25WU0101116, 25WU0101097, 25WU0101110, 25WU0101074

Section: CSE Rhinos

TABLE OF CONTENTS

1. Abstract	2
2. Introduction and Problem Statement	2
2.1 Introduction	2
2.2 Problem Statement	3
3. Objectives and Scope	3
3.1 Objectives	3
3.2 Scope of the Project	4
3.3 Scope Limitations	5
4. Software & Hardware Requirements	6
4.1 Software Requirements	6
4.2 System Requirements for Hardware	6
4.3 Development Environment	6
5. ER Diagram	7
5.1 Major Entities and Relationships	7
5.2 ER Diagram Representation	8
5.3 Important Relationships	8
6. Relational Schema and Normalization	9
6.1 Relational Schema	9
6.2 Primary Keys and Foreign Keys	10
6.3 Normalization	11
7. Data Dictionary	14
7.1 USER	15
7.2 DEPARTMENT	15
7.3 STUDENT	15
7.4 ACADEMIC_RECORD	15
7.5 SKILL	16
7.6 STUDENT_SKILL	16
7.7 COMPANY	16
7.8 JOB_PROFILE	16
7.9 PLACEMENT_DRIVE	17
7.10 ELIGIBILITY_CRITERIA	17
7.11 ELIGIBILITY_DEPARTMENTS	17
7.12 ELIGIBILITY_SKILLS	18
7.13 APPLICATION	18
7.14 SELECTION_ROUND	18
7.15 ROUND_RESULT	19
7.16 OFFER	19
8. SQL Commands Used	19

TABLE OF CONTENTS (continued)

8.1 DDL Commands	19
8.2 DML Commands	20
8.3 Sample Output	21
9. Queries with Outputs	22
9.1 Query 1 – Students with Departments	22
9.2 Query 2 – Companies and Job Profiles	23
9.3 Query 3 – Placement Drives with Company and Job Details	23
9.4 Query 4 – Application Count for Each Drive	24
9.5 Query 5 – Students with CGPA ≥ 8	24
9.6 Query 6 – Department-wise Student Count	25
9.7 Query 7 – Students Who Applied	25
9.8 Query 8 – High-CGPA Students	25
9.9 Query 9 – Applications with Company Names	26
9.10 Query 10 – Offers Issued	26
9.11 Query 11 – Company-wise Application Count	27
9.12 Query 12 – View-based Reporting	28
10. UI Design and Screenshots	28
10.1 Login Screen	28
10.2 Officer Dashboard	29
10.3 Student Dashboard	29
10.4 Student Management	30
10.5 Company Management	30
10.6 Job Profile Management	31
10.7 Placement Drive Management	31
10.8 Eligibility Configuration	31
10.9 Application Management	31
10.10 Selection Round Management	31
10.11 Offer Management	31
10.12 Student Offer Screen	32
10.13 Reports	32
10.14 Database Reflection	32
11. Implementation Details	33
11.1 Technology Stack	33
11.2 Application Architecture	33
11.3 Database and Business Logic	34
11.4 Transactions and Validation	34
11.5 Authentication and Reports	35
12. Testing	35
12.1 Functional Testing	35

TABLE OF CONTENTS (continued)

12.2 Database Testing	37
12.3 Transaction Testing	37
12.4 UI Testing	37
12.5 Test Results	38
13. Conclusion and Future Enhancements	38
13.1 Conclusion	38
13.2 Future Enhancements	38
14. References	39
15. Contribution of Each Member	39
16. GitHub Repository	39

LIST OF TABLES

Table 4.1: Software and Technology Requirements	6
Table 5.1: Important Relationships between Entities	9
Table 6.1: Primary Keys and Foreign Keys	10
Table 6.2: Unnormalized Form (UNF) – Sample Student Data	11
Table 6.3: First Normal Form (1NF) – Sample Student Data	12
Table 6.4: Second Normal Form (2NF) – STUDENT	12
Table 6.5: Second Normal Form (2NF) – STUDENT_SKILL	13
Table 6.6: Third Normal Form (3NF) – STUDENT	13
Table 6.7: Third Normal Form (3NF) – DEPARTMENT	13
Table 6.8: Third Normal Form (3NF) – COMPANY	14
Table 6.9: Third Normal Form (3NF) – SKILL	14
Table 6.10: Third Normal Form (3NF) – STUDENT_SKILL	14
Table 7.1: USER Table Data Dictionary	15
Table 7.2: DEPARTMENT Table Data Dictionary	15
Table 7.3: STUDENT Table Data Dictionary	15
Table 7.4: ACADEMIC_RECORD Table Data Dictionary	15
Table 7.5: SKILL Table Data Dictionary	16
Table 7.6: STUDENT_SKILL Table Data Dictionary	16
Table 7.7: COMPANY Table Data Dictionary	16
Table 7.8: JOB_PROFILE Table Data Dictionary	16
Table 7.9: PLACEMENT_DRIVE Table Data Dictionary	17
Table 7.10: ELIGIBILITY_CRITERIA Table Data Dictionary	17
Table 7.11: ELIGIBILITY_DEPARTMENTS Table Data Dictionary	17
Table 7.12: ELIGIBILITY_SKILLS Table Data Dictionary	18
Table 7.13: APPLICATION Table Data Dictionary	18
Table 7.14: SELECTION_ROUND Table Data Dictionary	18
Table 7.15: ROUND_RESULT Table Data Dictionary	19
Table 7.16: OFFER Table Data Dictionary	19
Table 11.1: Technology Stack	33
Table 12.1: Functional Testing – Test Cases	35
Table 12.2: Test Results	38
Table 15.1: Contribution of Each Member	39

LIST OF FIGURES

Figure 8.1: Database and Table Creation	22
Figure 8.2: DML Operation and Output	22
Figure 10.1: Application Login Screen	29
Figure 10.2: Placement Officer Dashboard	29
Figure 10.3: Student Dashboard	30
Figure 10.4: Student Management Screen	30
Figure 10.7: Placement Drive Management Screen	31
Figure 10.11: Officer Offer Management	32
Figure 10.13: Reports Screen	32
Figure 11.1: JDBC Database Connection	33
Figure 11.2: DAO and DML Database Operations	34
Figure 11.3: JDBC Transaction with COMMIT and ROLLBACK	35
Figure 12.1: Duplicate Student ID Test	36
Figure 12.2: Invalid CGPA Validation Test	36
Figure 12.3: Duplicate Application Test	36
Figure 12.4: Eligibility Validation Test	37

1.ABSTRACT

Campus Placement and Recruitment Drive Management System is a desktop application created to manage the entire process of campus placements in a college. The primary objective of the software is to centralize student data, records, skills, companies, job profiles, placement drives, eligibility criteria, applications, selection process, results, and offers. The software supports two user categories which are Placement Officer and Student. The placement officer will be able to manage students, companies, job profiles, placement drives, eligibility criteria, applications, selection rounds, offers, and generate reports. The student user category will be able to log in, view available placement drives, determine their eligibility, apply for appropriate drives, and accept or reject offers.

The system makes use of MySQL as the relational database and Java Swing as the desktop graphical interface. JDBC is used for connecting Java application to MySQL, whereas Maven will be used for project and dependencies management. The architecture of the application includes layers namely the UI layer, Service layer, DAO layer, JDBC layer, and Database layer.

The database is designed on the basis of Third Normal Form (3NF) with the help of primary keys, foreign keys, unique keys, validation, indexes, transaction, views and other database features. The system includes an eligibility engine in order to ensure that the students apply for the placement drives for which they fulfill the criteria.

Thus, it can be concluded that the system manages the placement process in a systematic manner and prevents duplication of information, manual verification, inaccurate records and problems in generation of reports.

2.Introduction and Problem Statement

2.1 Introduction

Placements are a critical aspect of university operations in that they help students interact with companies having various job opportunities. The whole placement process entails managing voluminous information regarding students, academic performance, skills, companies, job profiles, placement drive, eligibility criteria, application process, selections, result and offer letters. In the event that all these processes are conducted manually through different spreadsheets or records, there may arise a challenge in ensuring accuracy of the information. The placement cell has to keep on verifying the eligibility of the student in regard to certain companies or job profiles.

This Campus Placement and Recruitment Drive Management System is an application designed for the university placement cell. It takes care of the entire process of the placement starting from the registration and academic details of the students to checking their eligibility, making applications, selection process, result, and finally accepting/rejecting offers.

There are two types of users of this application: Placement Officer and Student. The placement officer will take care of all the data and processes related to placements, and the students will be able to see the right placement drives, check their eligibility, apply, see their selection process, and manage offers.

Java Swing is used to create the GUI and MySQL to implement the relational database. JDBC is used to establish the connection between the application and database. The application is implemented in such a way that the UI interacts with the Service Layer, which in turn interacts with DAO layer and eventually MySQL through JDBC.

2.2 Problem Statement

The manual handling of campus placement processes may give rise to many issues due to the distributed nature of the information and the multiple checks needed in various tasks. Some of the main issues are:

- Lack of centralization in keeping the student and academic information.
- Manual verification of student eligibility for various placement drives.
- Eligibility of an ineligible student applying for a drive.
- Duplicate applications of the same student for a drive.
- Keeping track of the selection round information.
- The risk of having duplicate or conflicting offer information.
- The ability to keep track of the complete placement history of a student.
- The ability to generate reports on applications, selections, offers, departments, and companies.

Therefore, there is a need for a centralized database management system which can store placement information, establish relations between entities, manage the data integrity, automate the eligibility verification process, and answer placement-related queries and generate reports. The suggested system can solve these issues by utilizing normalized relational database along with Java-based desktop application.

3. Objectives and Scope

3.1 Objectives

The key objective of Campus Placement and Recruitment Drive Management System is to develop an integrated database-based system that helps to manage the whole campus placement process. The objectives of the project are as follows:

1. To centralize placement information :

- To keep placement information regarding students, academics, skills, companies, job profile, placement drive, applications, selections, and offers.
2. To manage student information :
To maintain the student information along with the department, academic, and skill.
 3. To manage companies and job profiles :
To enable the placement officer to maintain information regarding companies and their job profiles.
 4. To manage the placement drives :
To maintain the information regarding the recruitment drive like the company, job profile, drive date, deadline, and status.
 5. To automate eligibility check :
To check if the student fulfills the required CGPA, backlog, department, year of graduation, and skill requirements before applying for the placement drive.
 6. For prevention of duplicate application:
For ensuring that a student cannot make more than one application for the same placement drive using database level constraints like UNIQUE(student_id, drive_id).
 7. For handling selection rounds & results :
For maintaining information regarding different stages of recruitment process including aptitude test, technical interview, and HR interview and their results.
 8. For handling placement offers :
For issuing placement offers by placement officers and accepting / rejecting them by the students.
 9. For integrity of data :
For using primary keys, foreign keys, unique constraints, NOT NULL, CHECK, DEFAULT, and indexes wherever required.
 10. For generating placement reports :
For generating various reports such as eligible students, applications, shortlist, offers, departmental placement, company results, and placement statistics.
 11. For creating normalized database :
For creating database as per Third Normal Form (3NF).

3.2 Scope of the Project

The scope of the system includes the main activities involved in handling the placement process at university campuses.

A. Student Management

The student details like student id, name, email, contact number, department, year of passing, and others can be saved in the system. The academic details like CGPA, marks, and backlogs will be kept separately.

B. Skill Management

The skills like Java, Python, SQL, Machine learning, React, and Communication can be handled separately. As there can be many-to-many relationship between student and skill, the STUDENT_SKILL association table will be used for handling the relation.

C. Company and Job Profile Management

The placement officers can handle the participating companies and the job profiles related to the company, like Software engineer, Data Analyst, Business analyst, and Backend Developer.

D. Placement Drive Management

The system is able to manage different placement drives individually and maintain details about them such as job profile, date of drive, deadline and status of drive.

E. Eligibility Management

The system is able to specify eligibility criteria for different placement drives, which may include minimum CGPA, maximum backlog, department, graduation year and skills needed.

F. Application Management

Students can apply to any of those placement drives for which they meet the eligibility criteria specified by the system. The system validates whether the drive is still live, deadline hasn't passed, the student is eligible to apply and if he hasn't applied previously.

G. Selection & Offer Management

The system is able to manage different rounds of selection and offer process and allow the students to either accept or reject their offer after being selected.

H. Reports

The system is capable of generating placement-based reports and statistics through the use of SQL statements like join, filter, aggregate, subquery, and view.

3.3 Scope Limitations

This particular project is implemented as a desktop application using Java Swing. Web technologies such as React, Node.js, Servlets, JSP, Spring, and REST APIs are not employed. MySQL is used as the database server and JDBC is used as database connectivity software.

4. Software & Hardware Requirements

4.1 Software Requirements

The below-listed software & technologies are required to build and execute the Campus Placement & Recruitment Drive Management System:

Table 4.1: Software and Technology Requirements

Software / Technology	Requirement / Purpose
Operating System	Windows / macOS / Linux
Java	Java 17 or above for application development and execution
Java Swing	Used to create the desktop graphical user interface
JDBC	Used to establish communication between Java and MySQL
MySQL	Used as the relational database management system
Maven	Used for dependency management and building the Java project
Git	Used for source-code version control
IDE / Code Editor	IntelliJ IDEA, Eclipse, VS Code, or any suitable Java IDE
SQL	Used for creating, manipulating, and querying the database

This project requires Java as the primary programming language, Java Swing for the GUI, JDBC for connecting to the database, Maven for dependency management, and MySQL as the database. The target Java version is Java 17+.

4.2 System Requirements for Hardware

There is no need for the user to possess advanced hardware as the software is a database application which is desktop-based. A normal computer that supports Java, Maven, and MySQL is enough.

Component Minimum Requirements :

- Processor Intel Core i3/ equivalent
- above RAM 4 GB and above
- Storage Must have at least 10 GB free space
- Screen Resolution 1366 x 768 pixels and above
- Keyboard Normal Keyboard Pointing Device Normal Mouse or Touchpad
- Internet Connectivity If the project requires Git/GitHub connection

4.3 Development Environment

The application is structured in layers as follows:

Swing UI → Service Layer → DAO Layer → JDBC → MySQL

Swing takes care of user interaction, the Service layer holds the business logic, the DAO layer performs database actions, while JDBC acts as the bridge between the Java code and MySQL. The project is developed using Maven, and the database can be set up via SQL scripts like `complete_database.sql`.

5.ER Diagram

The Entity Relationship (ER) diagram depicts the entities used in the Campus Placement and Recruitment Drive Management System along with the relationships among them. It is basically a logical view of the connection between the various components of the campus placement system. The important entities include Department, Student, Academic Record, Skill, Student Skill, Company, Job Profile, Placement Drive, Eligibility Criteria, Eligibility Departments, Eligibility Skills, Application, Selection Round, Round Result, Offer, and User.

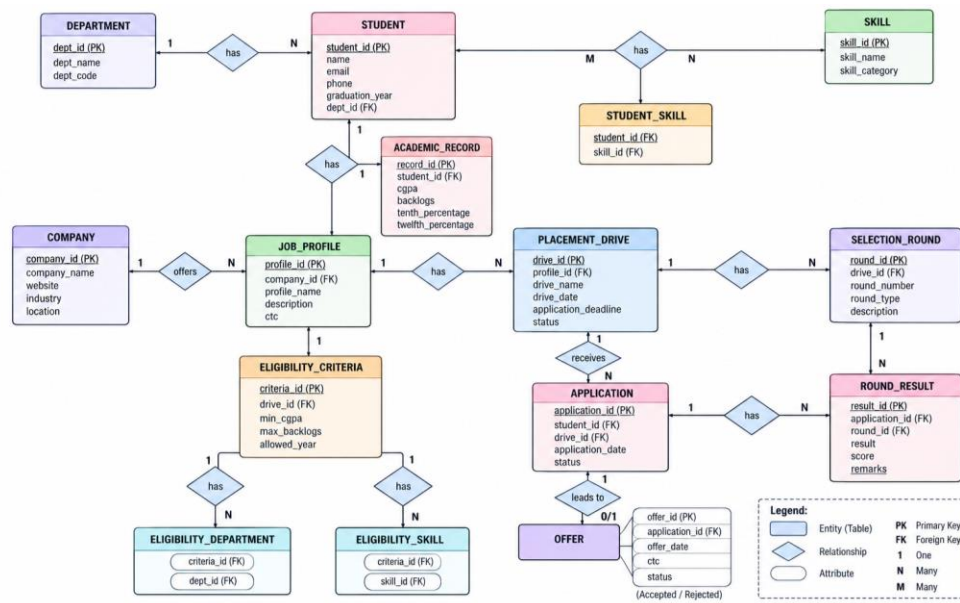
5.1 Major Entities and Relationships

The major relationships of the system are:

- One Department can have many Students.
- One Student has an Academic Record.
- One Student can have many Skills; also, one skill can have many students. The many to many relationship is represented by Student Skill.
- One Company can have many Job Profiles.
- One Job Profile can have many Placement Drives.
- One Placement Drive has its own Eligibility Criteria.
- One placement drive can have allowed Departments and Required Skills.
- One Student can apply for Placement Drive using Application entity.
- One placement drive can have many Selection Rounds.
- Every application can have Round Results for the different selection rounds.
- The application leads to Offer.
- The student then accepts or rejects the offer.

5.2 ER Diagram Representation

The overall relationship can be represented as follows:



5.3 Important Relationships

Table 5.1: Important Relationships between Entities

S. No.	Relationship	Cardinality	Description
1	Department → Student	1 : N	One department can have multiple students.
2	Student → Academic Record	1 : 1	Each student has one academic record.
3	Student ↔ Skill	M : N	A student can have multiple skills and a skill can belong to multiple students (implemented using STUDENT_SKILL).
4	Company → Job Profile	1 : N	One company can offer multiple job profiles.
5	Job Profile → Placement Drive	1 : N	One job profile can have multiple placement drives.
6	Placement Drive → Eligibility Criteria	1 : 1	Each placement drive has one eligibility criteria entry.
7	Eligibility Criteria → Eligibility Department	1 : N	One eligibility criteria can specify multiple allowed departments.
8	Eligibility Criteria → Eligibility Skill	1 : N	One eligibility criteria can specify multiple required skills.
9	Student → Application	1 : N	One student can apply to multiple placement drives.
10	Placement Drive → Application	1 : N	Multiple students can apply for one placement drive.
11	Placement Drive → Selection Round	1 : N	One placement drive can have multiple selection rounds.
12	Application → Round Result	1 : N	One application can have results for multiple rounds.
13	Selection Round → Round Result	1 : N	One selection round can have results for multiple applications.
14	Application → Offer	1 : 0/1	A successful application can result in one offer (optional).

The database structure has made a difference between entities such as department, student, company, job profile, and skills rather than keeping all the data in one table. This will help in reducing redundancy in the database of the project's 3NF.

6. Relational Schema and Normalization

6.1 Relational Schema

Relational schema is the representation of database in the form of tables, attributes, primary keys and foreign keys. The Campus Placement and Recruitment Drive Management System consists of separate related tables so that there is no duplication of information. The entities which have been considered for the project include Department, Student, Academic record, Skill, Student skill, Company, Job profile, Placement drive, Eligibility criteria, Eligibility department, Eligibility skill, Application, Selection round, Round result, Offer and User.

The relational schema of the **Campus Placement and Recruitment Drive Management System** is as follows:

DEPARTMENT : (dept_id, dept_name, dept_code)

STUDENT : (student_id, name, email, phone, graduation_year, dept_id)

ACADEMIC_RECORD: (record_id, student_id, cgpa, backlogs, tenth_percentage, twelfth_percentage)

SKILL : (skill_id, skill_name, skill_category)

STUDENT_SKILL : (student_id, skill_id)

COMPANY : (company_id, company_name, website, industry, location)

JOB_PROFILE : (profile_id, company_id, profile_name, description, etc)

PLACEMENT_DRIVE : (drive_id, profile_id, drive_name, drive_date, application_deadline, status)

ELIGIBILITY_CRITERIA : (criteria_id, drive_id, min_cgpa, max_backlogs, allowed_year)

ELIGIBILITY_DEPARTMENT : (criteria_id, dept_id)

ELIGIBILITY_SKILL : (criteria_id, skill_id)

APPLICATION : (application_id, student_id, drive_id, application_date, status)

SELECTION_ROUND : (round_id, drive_id, round_number, round_type, description)

ROUND_RESULT : (result_id, application_id, round_id, result, score, remarks)

OFFER : (offer_id, application_id, offer_date, etc, status)

USER : (user_id, username, password_hash, role, status)

6.2 Primary Keys and Foreign Keys

The primary key uniquely identifies each record in a table, while foreign keys establish relationships between related tables

Table 6.1: Primary Keys and Foreign Keys

Table	Primary Key	Foreign Key
DEPARTMENT	dept_id	—
STUDENT	student_id	dept_id → DEPARTMENT
ACADEMIC_RECORD	record_id	student_id → STUDENT
SKILL	skill_id	—
STUDENT_SKILL	student_id, skill_id	student_id → STUDENT; skill_id → SKILL
COMPANY	company_id	—
JOB_PROFILE	profile_id	company_id → COMPANY
PLACEMENT_DRIVE	drive_id	profile_id → JOB_PROFILE
ELIGIBILITY_CRITERIA	criteria_id	drive_id → PLACEMENT_DRIVE

Table 6.1: Primary Keys and Foreign Keys (continued)

ELIGIBILITY_DEPARTMENT	criteria_id, dept_id	criteria_id → ELIGIBILITY_CRITERIA; dept_id → DEPARTMENT
ELIGIBILITY_SKILL	criteria_id, skill_id	criteria_id → ELIGIBILITY_CRITERIA; skill_id → SKILL
APPLICATION	application_id	student_id → STUDENT; drive_id → PLACEMENT_DRIVE
SELECTION_ROUND	round_id	drive_id → PLACEMENT_DRIVE
ROUND_RESULT	result_id	application_id → APPLICATION; round_id → SELECTION_ROUND
OFFER	offer_id	application_id → APPLICATION
USER	user_id	—

Data integrity is maintained through the use of primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, DEFAULT, and indexes in the database. CGPA will be validated from 0-10, marks will always be within the valid range, and duplicate student-driver applications will not be allowed.

6.3 Normalization

The database is designed according to **Third Normal Form (3NF)**. Normalization is used to reduce data redundancy, avoid inconsistent data, and organize information into logically related tables.

Step 1: Unnormalized Form (UNF)

Consider the following sample data for four students:

Table 6.2: Unnormalized Form (UNF) – Sample Student Data

Student ID	Student Name	Department	Company	Skills
S101	Ananya	CSE	TCS	Java, SQL
S102	Rahul	CSE	Infosys	Python, SQL
S103	Priya	ECE	TCS	Java, Python
S104	Arjun	CSE	Wipro	Java, React

Here, the **Skills** column contains multiple values in a single cell. This violates the atomic-value requirement of 1NF.

Step 2: First Normal Form (1NF)

To convert the table into **1NF**, every attribute must contain a single atomic value.

Therefore, the multiple skills are separated into individual rows:

Table 6.3: First Normal Form (1NF) – Sample Student Data

Student ID	Student Name	Department	Company	Skill
S101	Ananya	CSE	TCS	Java
S101	Ananya	CSE	TCS	SQL
S102	Rahul	CSE	Infosys	Python
S102	Rahul	CSE	Infosys	SQL
S103	Priya	ECE	TCS	Java
S103	Priya	ECE	TCS	Python
S104	Arjun	CSE	Wipro	Java
S104	Arjun	CSE	Wipro	React

Now every cell contains only **one value**, so the relation satisfies **1NF**.

However, there is still a lot of repeated information. For example, Ananya's name, department, and company are repeated for each skill.

Step 3: Second Normal Form (2NF)

For this example, the combined key can be considered:

(Student ID, Skill)

The problem is that attributes such as:

- Student Name
- Department
- Company

depend only on **Student ID**, not on the complete (Student ID, Skill) key.

This is called a **partial dependency**.

Therefore, we separate the student information from the student-skill relationship.

STUDENT

Table 6.4: Second Normal Form (2NF) – STUDENT

Student ID	Student Name	Department	Company
S101	Ananya	CSE	TCS
S102	Rahul	CSE	Infosys
S103	Priya	ECE	TCS
S104	Arjun	CSE	Wipro

STUDENT_SKILL

Table 6.5: Second Normal Form (2NF) – STUDENT_SKILL

Student ID	Skill
S101	Java
S101	SQL
S102	Python
S102	SQL
S103	Java
S103	Python
S104	Java
S104	React

Now the student details depend only on **Student ID**, while the student-skill relationship is maintained separately.

Thus, the partial dependency is removed and the design satisfies **2NF**.

Step 4: Third Normal Form (3NF)

Although the previous design is better, there can still be a **transitive dependency**.

For example:

Student ID → Department
 Department → Department details

Similarly:

Student ID → Company
 Company → Company details

Therefore, department and company information should be stored separately.

STUDENT

Table 6.6: Third Normal Form (3NF) – STUDENT

Student ID	Student Name	Department ID	Company ID
S101	Ananya	D01	C01
S102	Rahul	D01	C02
S103	Priya	D02	C01
S104	Arjun	D01	C03

DEPARTMENT

Table 6.7: Third Normal Form (3NF) – DEPARTMENT

Department ID	Department Name
D01	Computer Science and Engineering

Table 6.7: Third Normal Form (3NF) – DEPARTMENT (continued)

D02	Electronics and Communication Engineering
-----	---

COMPANY

Table 6.8: Third Normal Form (3NF) – COMPANY

Company ID	Company Name
C01	TCS
C02	Infosys
C03	Wipro

SKILL

Table 6.9: Third Normal Form (3NF) – SKILL

Skill ID	Skill Name
SK01	Java
SK02	SQL
SK03	Python
SK04	React

STUDENT_SKILL

Table 6.10: Third Normal Form (3NF) – STUDENT_SKILL

Student ID	Skill ID
S101	SK01
S101	SK02
S102	SK03
S102	SK02
S103	SK01
S103	SK03
S104	SK01
S104	SK04

Now the information is properly divided into independent entities.

7. Data Dictionary

The data dictionary describes the structure of the database tables, their attributes, data types, and constraints. It helps in understanding what type of information is stored in each table and how the tables maintain data consistency. The following data dictionary is based on the implemented database structure.

7.1 USER

The USER table stores the login and account information of users who access the system.

Table 7.1: USER Table Data Dictionary

Attribute	Description	Type	Constraints
user_id	Unique user identifier	INT	PK
username	Login username	VARCHAR	UNIQUE, NOT NULL
password_hash	Hashed password	VARCHAR	NOT NULL
role	User role	VARCHAR/ENUM	NOT NULL
status	Account status	VARCHAR/ENUM	DEFAULT/NOT NULL

7.2 DEPARTMENT

The DEPARTMENT table stores the departments available in the university.

Table 7.2: DEPARTMENT Table Data Dictionary

Attribute	Description	Type	Constraints
department_id	Department identifier	INT	PK
department_name	Name of department	VARCHAR	UNIQUE, NOT NULL

7.3 STUDENT

The STUDENT table stores the basic details of students participating in placement activities.

Table 7.3: STUDENT Table Data Dictionary

Attribute	Description	Type	Constraints
student_id	Student identifier	VARCHAR	PK
name	Student name	VARCHAR	NOT NULL
email	Student email	VARCHAR	UNIQUE, NOT NULL
phone	Contact number	VARCHAR	—
department_id	Student department	INT	FK, NOT NULL
graduation_year	Expected graduation year	INT	NOT NULL

7.4 ACADEMIC_RECORD

The ACADEMIC_RECORD table stores the academic information required for placement eligibility.

Table 7.4: ACADEMIC_RECORD Table Data Dictionary

Attribute	Description	Type	Constraints
academic_record_id	Record identifier	INT	PK
student_id	Associated student	VARCHAR	FK, UNIQUE
cgpa	Student CGPA	DECIMAL	CHECK 0–10

Table 7.4: ACADEMIC_RECORD Table Data Dictionary (continued)

marks	Academic marks	DECIMAL	CHECK 0–100
backlogs	Number of backlogs	INT	CHECK >= 0

7.5 SKILL

The SKILL table stores the different technical or other skills that can be associated with students and placement eligibility requirements.

Table 7.5: SKILL Table Data Dictionary

Attribute	Description	Type	Constraints
skill_id	Skill identifier	INT	PK
skill_name	Skill name	VARCHAR	UNIQUE, NOT NULL

7.6 STUDENT_SKILL

The STUDENT_SKILL table connects students with their skills. Since a student can have multiple skills and a skill can belong to multiple students, this table represents a many-to-many relationship.

Table 7.6: STUDENT_SKILL Table Data Dictionary

Attribute	Description	Constraints
student_id	Student	PK, FK
skill_id	Skill	PK, FK

Composite Primary Key: (student_id, skill_id)

7.7 COMPANY

The COMPANY table stores the details of companies participating in the placement process.

Table 7.7: COMPANY Table Data Dictionary

Attribute	Description	Constraints
company_id	Company identifier	PK
company_name	Company name	UNIQUE, NOT NULL
industry	Industry/category	—
website	Company website	—
contact_email	Contact email	—
contact_phone	Contact number	—

7.8 JOB_PROFILE

The JOB_PROFILE table stores the job positions offered by companies.

Table 7.8: JOB_PROFILE Table Data Dictionary

Attribute	Description	Constraints
job_profile_id	Job profile identifier	PK

Table 7.8: JOB_PROFILE Table Data Dictionary (continued)

company_id	Company	FK, NOT NULL
title	Job title	NOT NULL
description	Job description	—
location	Job location	—
salary_package	Package offered	—

7.9 PLACEMENT_DRIVE

The PLACEMENT_DRIVE table stores information about individual placement drives conducted for job profiles.

Table 7.9: PLACEMENT_DRIVE Table Data Dictionary

Attribute	Description	Constraints
drive_id	Drive identifier	PK
job_profile_id	Associated job	FK
drive_date	Recruitment date	NOT NULL
application_deadline	Last application date	NOT NULL
status	Drive status	NOT NULL

7.10 ELIGIBILITY_CRITERIA

The ELIGIBILITY_CRITERIA table stores the basic academic requirements that students must satisfy for a particular placement drive.

Table 7.10: ELIGIBILITY_CRITERIA Table Data Dictionary

Attribute	Description	Constraints
eligibility_id	Eligibility identifier	PK
drive_id	Placement drive	FK, UNIQUE
min_cgpa	Minimum CGPA	CHECK
max_backlogs	Maximum backlogs	CHECK
graduation_year	Required graduation year	—

7.11 ELIGIBILITY_DEPARTMENTS

The ELIGIBILITY_DEPARTMENTS table connects eligibility criteria with the departments allowed to participate in a placement drive.

Table 7.11: ELIGIBILITY_DEPARTMENTS Table Data Dictionary

Attribute	Description	Constraints
eligibility_id	Eligibility record	PK, FK

Table 7.11: ELIGIBILITY_DEPARTMENTS Table Data Dictionary (continued)

department_id	Eligible department	PK, FK
---------------	---------------------	--------

7.12 ELIGIBILITY_SKILLS

The ELIGIBILITY_SKILLS table stores the skills required for a particular placement drive.

Table 7.12: ELIGIBILITY_SKILLS Table Data Dictionary

Attribute	Description	Constraints
eligibility_id	Eligibility record	PK, FK
skill_id	Required skill	PK, FK

7.13 APPLICATION

The APPLICATION table stores the applications submitted by students for placement drives.

Table 7.13: APPLICATION Table Data Dictionary

Attribute	Description	Constraints
application_id	Application identifier	PK
student_id	Applicant	FK
drive_id	Placement drive	FK
application_date	Application timestamp/date	NOT NULL
status	Application status	NOT NULL

Important Constraint:

UNIQUE(student_id, drive_id)

This prevents the same student from applying more than once to the same placement drive.

7.14 SELECTION_ROUND

The SELECTION_ROUND table stores the different rounds conducted as part of a placement drive.

Table 7.14: SELECTION_ROUND Table Data Dictionary

Attribute	Description	Constraints
round_id	Round identifier	PK
drive_id	Placement drive	FK
round_name	Round name	NOT NULL
sequence_no	Round order	NOT NULL
round_date	Date of round	—

Important Constraint:

UNIQUE(drive_id, sequence_no)

This ensures that two rounds of the same placement drive cannot have the same sequence number.

7.15 ROUND_RESULT

The ROUND_RESULT table stores the result of each selection round for an application.

Table 7.15: ROUND_RESULT Table Data Dictionary

Attribute	Description	Constraints
result_id	Result identifier	PK
application_id	Application	FK
round_id	Selection round	FK
result	Round outcome	NOT NULL
remarks	Officer remarks	—

Important Constraint:

UNIQUE(application_id, round_id)

This ensures that only one result is recorded for an application in a particular selection round.

7.16 OFFER

The OFFER table stores the details of placement offers given to selected students.

Table 7.16: OFFER Table Data Dictionary

Attribute	Description	Constraints
offer_id	Offer identifier	PK
application_id	Selected application	FK, UNIQUE
offer_date	Date issued	NOT NULL
package	Offered package	—
status	Offer status	NOT NULL

8. SQL COMMANDS USED

This section demonstrates the SQL commands used to create, modify, and manage the database of the Campus Placement and Recruitment Drive Management System. The project uses both **DDL (Data Definition Language)** and **DML (Data Manipulation Language)** commands.

8.1 DDL Commands

DDL commands are used to define and modify the structure of the database.

The main DDL commands used in the project are:

- CREATE DATABASE
- CREATE TABLE
- ALTER TABLE
- CREATE INDEX
- CREATE VIEW
- CREATE PROCEDURE
- CREATE TRIGGER

Example 1: Database Creation

```
CREATE DATABASE IF NOT EXISTS campus_placement;
```

Example 2: Table Creation

```
CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE
);
```

Example 3: Foreign Key

```
FOREIGN KEY (department_id)
REFERENCES department(department_id);
```

Example 4: Unique Constraint

```
UNIQUE (student_id, drive_id);
```

This prevents a student from applying to the same placement drive more than once.

The project also uses indexes, views, stored procedures, and triggers to support efficient searching, reporting, and database operations

8.2 DML Commands

DML commands are used to insert, modify, delete, and retrieve data from the database.

The main DML commands used are:

- INSERT
- UPDATE
- DELETE
- SELECT

INSERT:

```
INSERT INTO department  
(department_id, department_name)
```

VALUES:

```
(1, 'Computer Science and Engineering');
```

UPDATE:

```
UPDATE placement_drive  
SET status = 'CLOSED'  
WHERE drive_id = 1;
```

DELETE:

```
DELETE FROM skill  
WHERE skill_id = 5;
```

SELECT:

```
SELECT *  
FROM student;
```

These commands are used throughout the application for managing student, company, placement drive, application, selection, and offer data.

8.3 Sample Output

For the important SQL operations, the report should include:

SQL Command → MySQL Screenshot → Output

For example:

```
SELECT student_id, name, email  
FROM student;
```

1. Database and Table Creation

```
01. Database + Table Creation
database/01_create_database.sql & 02_create_tables.sql (MySQL 8.0 DDL)

1  -- Campus Placement and Recruitment Drive Management System
2  DROP DATABASE IF EXISTS campus_placement;
3  CREATE DATABASE campus_placement CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
4  USE campus_placement;
5
6  CREATE TABLE students (
7      student_id VARCHAR(20) PRIMARY KEY,
8      user_id INT NULL,
9      full_name VARCHAR(100) NOT NULL,
10     email VARCHAR(120) NOT NULL,
11     phone VARCHAR(15) NULL,
12     dept_id INT NOT NULL,
13     graduation_year SMALLINT NOT NULL,
14     cgpa DECIMAL(4,2) NOT NULL DEFAULT 0.00,
15     backlogs TINYINT UNSIGNED NOT NULL DEFAULT 0,
16     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
17 );
18
19 CREATE TABLE placement_drives (
20     drive_id INT AUTO_INCREMENT PRIMARY KEY,
21     job_id INT NOT NULL,
22     drive_date DATE NOT NULL,
23     application_deadline DATE NOT NULL,
24     venue VARCHAR(100) NOT NULL,
25     status ENUM('UPCOMING','OPEN','IN_PROGRESS','COMPLETED','CANCELLED') NOT NULL DEFAULT 'UPCOMING'
26 );
```

Figure 8.1: Database and Table Creation

2. DML Operation and Output

```
07. Database View for Dynamic Pipeline Analytics
database/07_views.sql

1  -- vw_student_placement_summary: consolidated real-time profile analytics
2  CREATE OR REPLACE VIEW vw_student_placement_summary AS
3  SELECT s.student_id, s.full_name, dp.dept_code, s.cgpa, s.backlogs,
4         COUNT(DISTINCT a.application_id) AS total_applications,
5         COUNT(DISTINCT CASE WHEN a.status = 'SHORTLISTED' THEN a.application_id END) AS shortlisted_count,
6         COUNT(DISTINCT CASE WHEN a.status = 'SELECTED' THEN a.application_id END) AS selections_count,
7         COUNT(DISTINCT o.offer_id) AS offers_received,
8         MAX(o.package_lpa) AS highest_offer_lpa,
9         CASE WHEN COUNT(DISTINCT CASE WHEN o.status = 'ACCEPTED' THEN o.offer_id END) > 0
10             THEN 'PLACED'
11             ELSE 'UNPLACED'
12         END AS placement_status
13 FROM students s
14 JOIN departments dp ON dp.dept_id = s.dept_id
15 LEFT JOIN applications a ON a.student_id = s.student_id
16 LEFT JOIN offers o ON o.application_id = a.application_id
17 GROUP BY s.student_id, s.full_name, dp.dept_code, s.cgpa, s.backlogs;
```

Figure 8.2 : DML Operation and Output

9. QUERIES WITH OUTPUTS

The database supports different types of SQL queries such as joins, aggregate functions, filtering, nested queries, and view-based reporting. The following queries demonstrate the major database operations used in the project. The outputs shown in the report should be taken from the actual MySQL execution.

9.1 Query 1 – Students with Departments

```
SELECT
    s.student_id,
    s.name,
    d.department_name
```

FROM student s
 JOIN department d
 ON s.department_id = d.department_id;

Concept: INNER JOIN

```

Query 1 - Students - Departments Multi-table INNER JOIN connecting Students entity to Departments master table.

mysql> USE campus_placement;
Database changed
mysql> SELECT s.student_id, s.full_name, s.email, d.dept_name, s.gpa, s.backlogs
  -> FROM students s
  -> JOIN departments d ON s.dept_id = d.dept_id
  -> WHERE d.dept_name LIKE '%cs%';

+-----+-----+-----+-----+-----+-----+
| student_id | full_name | email | dept_name | gpa | backlogs |
+-----+-----+-----+-----+-----+-----+
| 21150001 | Ananya Rao | ananya.rao@university.edu | Computer Science and Engineering | 9.12 | 0 |
| 21150002 | Rahul Sharma | rahul.sharma@university.edu | Computer Science and Engineering | 8.45 | 0 |
| 21150003 | Priya Sharma | priya.sharma@university.edu | Computer Science and Engineering | 7.66 | 1 |
| 21150004 | Karthik Reddy | karthik.reddy@university.edu | Computer Science and Engineering | 8.88 | 0 |
| 21150005 | Susha Iyer | susha.iyer@university.edu | Computer Science and Engineering | 6.90 | 2 |
| 21150006 | Arjun Rao | arjun.rao@university.edu | Computer Science and Engineering | 9.40 | 0 |
| 21150007 | Aditya Kulkarni | aditya.kulkarni@university.edu | Electronics and Communication Engineering | 8.30 | 0 |
| 21150008 | Nisha Verma | nisha.verma@university.edu | Electronics and Communication Engineering | 7.45 | 0 |
+-----+-----+-----+-----+-----+-----+

P: Query (2) set (0.02 sec)

[REV CONCEPT] Relational Concept Foreign Key Navigation [students.dept_id <- departments.dept_id]
  
```

9.2 Query 2 – Companies and Job Profiles

SELECT
 c.company_name,
 jp.title,
 jp.location
 FROM company c
 JOIN job_profile jp
 ON c.company_id = jp.company_id;

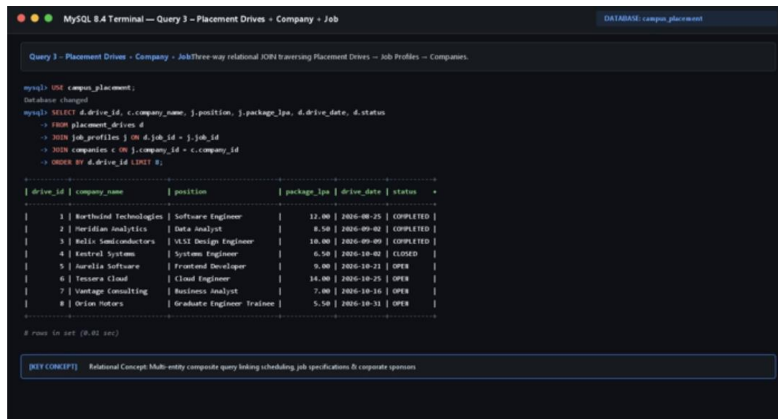
Concept: JOIN

9.3 Query 3 – Placement Drives with Company and Job Details

SELECT
 pd.drive_id,
 c.company_name,
 jp.title,
 pd.drive_date,
 pd.application_deadline,
 pd.status
 FROM placement_drive pd
 JOIN job_profile jp
 ON pd.job_profile_id = jp.job_profile_id
 JOIN company c
 ON jp.company_id = c.company_id;

Concept: Multi-table JOIN

Output:



```
mysql> USE campus_placement;
Database changed
mysql> SELECT d.drive_id, c.company_name, j.position, j.package_lpa, d.drive_date, d.status
-> FROM placement_drives d
-> JOIN job_profiles j ON d.job_id = j.job_id
-> JOIN companies c ON j.company_id = c.company_id
-> ORDER BY d.drive_id LIMIT 8;
```

drive_id	company_name	position	package_lpa	drive_date	status
1	Northind Technologies	Software Engineer	12.00	2020-09-25	COMPLETED
2	Horizon Analytics	Data Analyst	8.50	2020-09-02	COMPLETED
3	Relix Semiconductors	VLSI Design Engineer	10.00	2020-09-09	COMPLETED
4	Kontrol Systems	System Engineer	6.50	2020-10-02	CLOSED
5	Aurelia Software	Frontend Developer	9.00	2020-10-21	OPEN
6	Tenara Cloud	Cloud Engineer	10.00	2020-10-25	OPEN
7	Vantage Consulting	Business Analyst	7.00	2020-10-16	OPEN
8	Orion Motors	Graduate Engineer Trainee	5.50	2020-10-31	OPEN

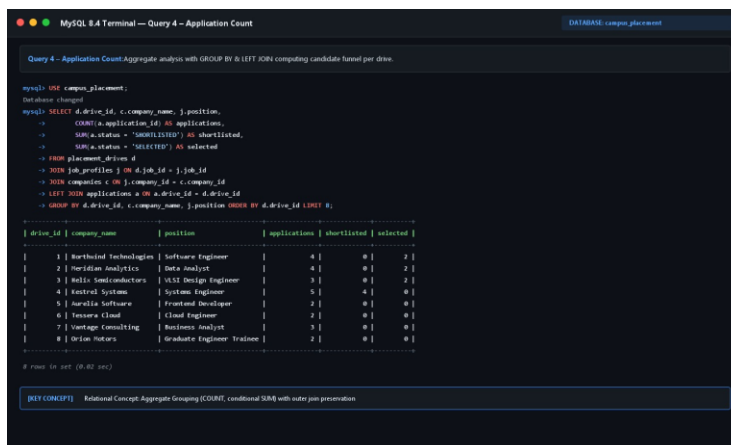
8 rows (in set) (0.02 sec)

[KEY CONCEPT] Relational Concept: Multi entity composite query linking scheduling job specifications to composite sponsors

9.4 Query 4 – Application Count for Each Drive

```
SELECT
    pd.drive_id,
    COUNT(a.application_id) AS application_count
FROM placement_drive pd
LEFT JOIN application a
ON pd.drive_id = a.drive_id
GROUP BY pd.drive_id;
```

Concept: LEFT JOIN, COUNT, GROUP BY



```
mysql> USE campus_placement;
Database changed
mysql> SELECT d.drive_id, c.company_name, j.position,
-> COUNT(a.application_id) AS applications,
-> SUM(a.status = 'SHORTLISTED') AS shortlisted,
-> SUM(a.status = 'SELECTED') AS selected
-> FROM placement_drives d
-> JOIN job_profiles j ON d.job_id = j.job_id
-> JOIN companies c ON j.company_id = c.company_id
-> LEFT JOIN applications a ON a.drive_id = d.drive_id
-> GROUP BY d.drive_id, c.company_name, j.position ORDER BY d.drive_id LIMIT 8;
```

drive_id	company_name	position	applications	shortlisted	selected
1	Northind Technologies	Software Engineer	4	0	2
2	Horizon Analytics	Data Analyst	4	0	2
3	Relix Semiconductors	VLSI Design Engineer	3	0	1
4	Kontrol Systems	System Engineer	5	1	0
5	Aurelia Software	Frontend Developer	2	0	0
6	Tenara Cloud	Cloud Engineer	2	0	0
7	Vantage Consulting	Business Analyst	3	0	0
8	Orion Motors	Graduate Engineer Trainee	2	0	0

8 rows (in set) (0.02 sec)

[KEY CONCEPT] Relational Concept: Aggregate Grouping (COUNT, conditional SUM) with outer join presentation

9.5 Query 5 – Students with CGPA ≥ 8

```
SELECT
    s.student_id,
```

```
s.name,  
ar.cgpa  
FROM student s  
JOIN academic_record ar  
ON s.student_id = ar.student_id  
WHERE ar.cgpa >= 8.0;
```

Concept: JOIN and filtering

9.6 Query 6 – Department-wise Student Count

```
SELECT  
    d.department_name,  
    COUNT(s.student_id) AS student_count  
FROM department d  
LEFT JOIN student s  
ON d.department_id = s.department_id  
GROUP BY d.department_id, d.department_name;
```

Concept: Aggregate function

9.7 Query 7 – Students Who Applied

```
SELECT  
    s.student_id,  
    s.name,  
    COUNT(a.application_id) AS applications  
FROM student s  
JOIN application a  
ON s.student_id = a.student_id  
GROUP BY s.student_id, s.name;
```

Concept: JOIN and GROUP BY

9.8 Query 8 – High-CGPA Students

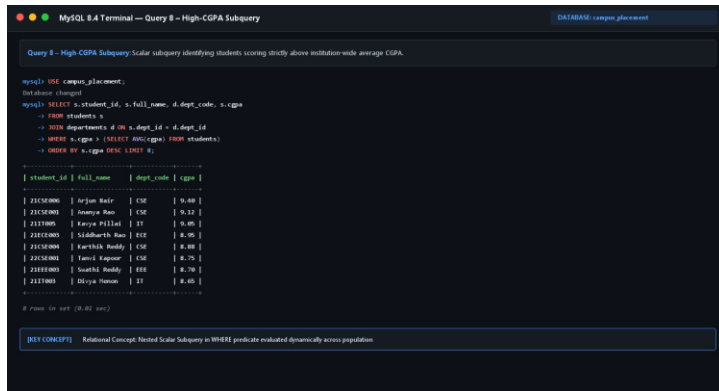
```
SELECT student_id, name  
FROM student  
WHERE student_id IN (  
    SELECT student_id
```

```

FROM academic_record
WHERE cgpa >= 8.5
);

```

Concept: Nested query / Subquery



The screenshot shows a MySQL terminal window titled "MySQL 8.4 Terminal - Query 8 - High-CGPA Subquery". The database is "campus_placement". The query being executed is:

```

mysql> USE campus_placement;
Database changed
mysql> SELECT s.student_id, s.full_name, d.dept_code, s.cgpa
-> FROM students s
-> JOIN departments d ON s.dept_id = d.dept_id
-> WHERE s.cgpa > (SELECT AVG(cgpa) FROM students)
-> ORDER BY s.cgpa DESC LIMIT 5;

```

The results are displayed in a table:

student_id	full_name	dept_code	cgpa
21150000	Aarjun Bhat	CSE	9.00
21150001	Ananya Rao	CSE	9.12
21150002	Ranya Pillai	IT	9.00
21150003	Siddharth Rao	ECE	8.95
21150004	Karthik Reddy	CSE	8.80
21150005	Tanvi Kulkarni	CSE	8.75
21150006	Sumith Reddy	ECE	8.70
21150007	Divya Nandan	IT	8.65

At the bottom, it says "8 rows in set (0.02 sec)".

9.9 Query 9 – Applications with Company Names

```

SELECT
    s.name AS student_name,
    c.company_name,
    jp.title AS job_profile,
    a.status
FROM application a
JOIN student s
ON a.student_id = s.student_id
JOIN placement_drive pd
ON a.drive_id = pd.drive_id
JOIN job_profile jp
ON pd.job_profile_id = jp.job_profile_id
JOIN company c
ON jp.company_id = c.company_id;

```

Concept: Multiple JOINS

9.10 Query 10 – Offers Issued

```

SELECT
    s.student_id,
    s.name,

```

```

    c.company_name,
    jp.title,
    o.package,
    o.status
FROM offer o
JOIN application a
ON o.application_id = a.application_id
JOIN student s
ON a.student_id = s.student_id
JOIN placement_drive pd
ON a.drive_id = pd.drive_id
JOIN job_profile jp
ON pd.job_profile_id = jp.job_profile_id
JOIN company c
ON jp.company_id = c.company_id;

```

Concept: Multiple JOINS

The screenshot shows a MySQL 8.4 Terminal window with the title 'Query 10 - Offers Issued'. The query being executed is a complex JOIN query that filters for offers issued to students, through applications, placement drives, and job profiles, and then joins with the company table. The results are displayed in a table format with columns: offer_id, full_name, company_name, position, package_lpa, status, and offer_date. The results show 6 rows of data.

```

mysql> USE campus_placement;
Database changed
mysql> SELECT o.offer_id, s.full_name, c.company_name, j.position, a.package_lpa, o.status, o.offer_date
-> FROM offers o
-> JOIN applications a ON a.application_id = o.application_id
-> JOIN students s ON s.student_id = a.student_id
-> JOIN placement_drives d ON a.drive_id = d.drive_id
-> JOIN job_profiles j ON d.job_id = j.job_id
-> JOIN companies c ON j.company_id = c.company_id
-> WHERE o.offer_id < 10;
+-----+-----+-----+-----+-----+-----+
| offer_id | full_name | company_name | position | package_lpa | status | offer_date |
+-----+-----+-----+-----+-----+-----+
| 1 | Ananya Rao | Northind Technologies | Software Engineer | 12.00 | ACCEPTED | 2020-08-10 |
| 2 | Rahul Mehta | Northind Technologies | Software Engineer | 12.00 | REJECTED | 2020-08-10 |
| 3 | Priya Sharma | Meridian Analytics | Data Analyst | 8.50 | PENDING | 2020-09-06 |
| 4 | Arjun Katar | Meridian Analytics | Data Analyst | 8.50 | ACCEPTED | 2020-09-06 |
| 5 | Aditya Kulkarni | Nuclei Semiconductors | VLSI Design Engineer | 18.00 | ACCEPTED | 2020-09-14 |
| 6 | Siddharth Rao | Nuclei Semiconductors | VLSI Design Engineer | 18.00 | PENDING | 2020-09-14 |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.02 sec)

```

9.11 Query 11 – Company-wise Application Count

```

SELECT
    c.company_name,
    COUNT(a.application_id) AS total_applications
FROM company c
JOIN job_profile jp
ON c.company_id = jp.company_id
JOIN placement_drive pd
ON jp.job_profile_id = pd.job_profile_id
LEFT JOIN application a

```

```
ON pd.drive_id = a.drive_id
GROUP BY c.company_id, c.company_name;
```

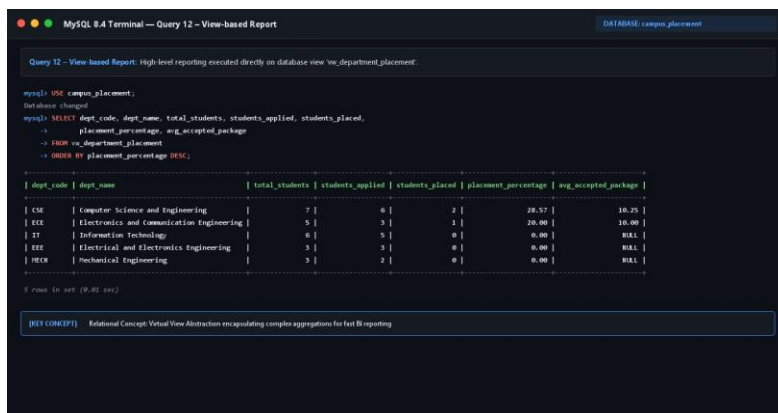
Concept: JOIN, COUNT and GROUP BY

9.12 Query 12 – View-based Reporting

The project also uses database views to provide reusable reports combining information such as students, applications, drives, companies, and job profiles.

```
SELECT *
FROM placement_report;
```

Concept: Database VIEW



Query 12 - View-based Report: High-level reporting executed directly on database view 'view_department_placement'.

```
mysql> USE campus_placement;
Database changed
mysql> SELECT dept_code, dept_name, total_students, students_applied, students_placed,
-> placement_percentage, avg_accepted_package
-> FROM view_department_placement
-> GROUP BY dept_code, dept_name HASC;
```

dept_code	dept_name	total_students	students_applied	students_placed	placement_percentage	avg_accepted_package
CSE	Computer Science and Engineering	7	6	2	28.57	10.25
ECE	Electronics and Communication Engineering	5	3	2	20.00	10.00
IT	Information Technology	6	5	0	0.00	N/A
ECE	Electrical and Electronics Engineering	3	3	0	0.00	N/A
MECH	Mechanical Engineering	3	2	0	0.00	N/A

5 rows (4.00 sec)

[KEY CONCEPT] Relational Concept: Virtual View Abstraction encapsulating complex aggregations for fast BI reporting

10. UI DESIGN AND SCREENSHOTS

The project uses a **Java Swing desktop interface** connected to the MySQL database through JDBC. The interface follows a role-based design, providing different functionality for placement officers and students.

10.1 Login Screen

The login screen allows users to enter their username and password. After successful authentication, the system identifies the user's role and redirects them to the appropriate dashboard.

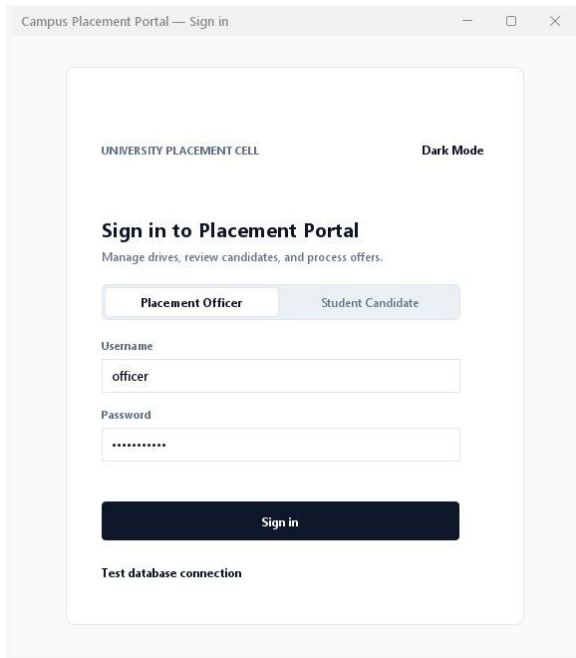


Figure 10.1: Application Login Screen

10.2 Officer Dashboard

The officer dashboard provides an overview of placement activities such as students, companies, placement drives, applications, and offers.

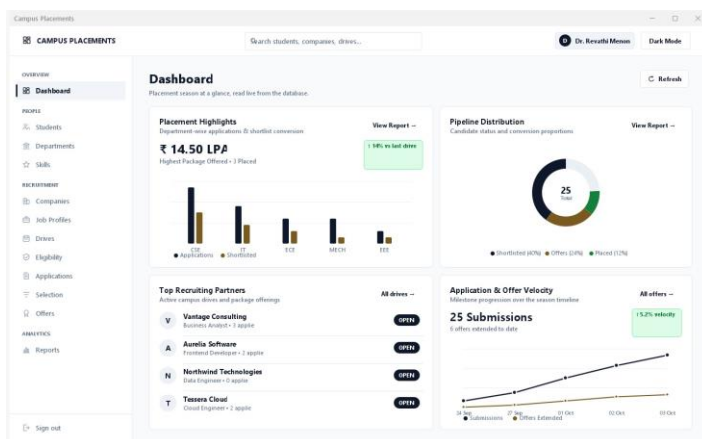


Figure 10.2: Placement Officer Dashboard

10.3 Student Dashboard

The student dashboard provides access to placement drives, applications, selection status, offers, and profile information.

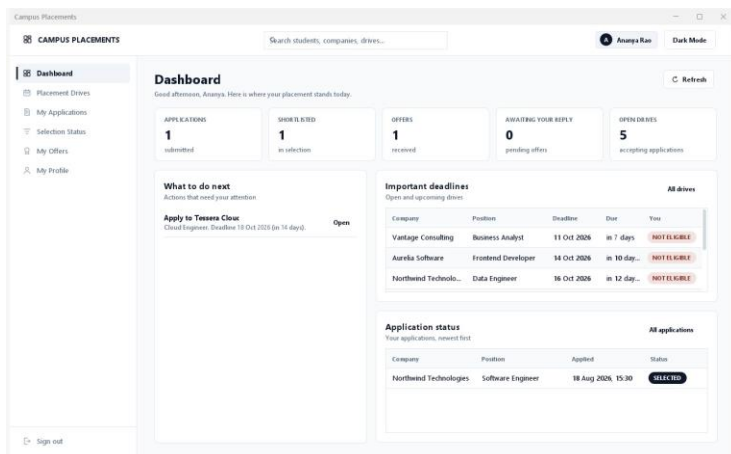


Figure 10.3: Student Dashboard

10.4 Student Management

The student management screen allows the placement officer to view and manage student information.

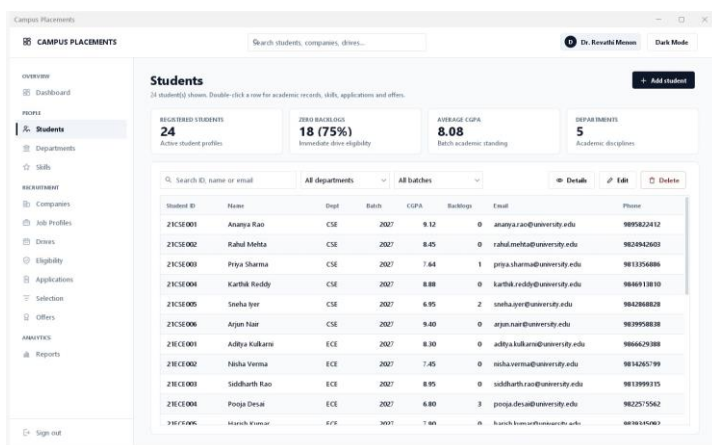


Figure 10.4: Student Management Screen

10.5 Company Management

The company management screen is used to manage companies participating in the placement process.

10.6 Job Profile Management

The job profile screen allows officers to create and manage job profiles associated with companies.

10.7 Placement Drive Management

The placement drive screen allows officers to create drives, set dates and deadlines, assign job profiles, and update drive status.

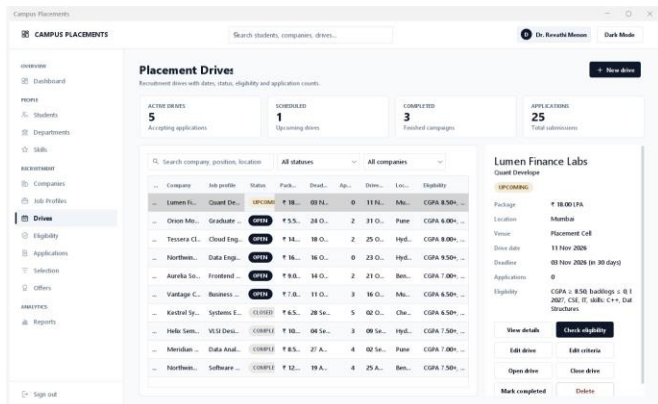


Figure 10.7: Placement Drive Management Screen

10.8 Eligibility Configuration

The eligibility screen allows officers to configure minimum CGPA, maximum backlogs, graduation year, eligible departments, and required skills.

10.9 Application Management

The application screen allows the officer to view and manage student applications for placement drives.

10.10 Selection Round Management

The selection module allows officers to create selection rounds, set their sequence, view candidates, and record results.

10.11 Offer Management

The officer can issue placement offers to selected candidates through the offer management screen.

#	Student	Dept	Company	Position	Package	Offer date	Answer date	Status	Responded
6	Sakshith Ran	ECE	Hella Semiconductors	VLSI Design Engineer	₹ 10.00 LPA	18 Sept 20...	03 Jan 20...	PENDING	---
5	Aditya Kulkarni	ECE	Hella Semiconductors	VLSI Design Engineer	₹ 10.00 LPA	14 Sept 20...	03 Jan 20...	ACCEPTED	18 Sept 2020, 16...
4	Arjun Nair	CSE	Meridian Analytics	Data Analyst	₹ 8.50 LPA	06 Sept 20...	03 Jan 20...	ACCEPTED	10 Sept 2020, 16...
3	Priya Sharma	CSE	Meridian Analytics	Data Analyst	₹ 8.50 LPA	06 Sept 20...	03 Jan 20...	PENDING	---
2	Rahul Mehra	CSE	Northwind Techno...	Software Engineer	₹ 12.00 LPA	30 Aug 20...	03 Jan 20...	REJECTED	03 Sept 2020, 16...
1	Ananya Rao	CSE	Northwind Techno...	Software Engineer	₹ 12.00 LPA	30 Aug 20...	03 Jan 20...	ACCEPTED	03 Sept 2020, 16...

Figure 10.11: Officer Offer Management

10.12 Student Offer Screen

Students can view the placement offers received through their dashboard.

10.13 Reports

The reports screen provides access to placement-related reports generated from the database.

Drive	Company	Position	Student ID	Student	Dept
1	Northwind Tech...	Software Engine...	21CSE001	Ananya Rao	CSE
1	Northwind Tech...	Software Engine...	21CSE004	Karthik Reddy	CSE
1	Northwind Tech...	Software Engine...	21IT003	Divya Menon	IT
1	Northwind Tech...	Software Engine...	21CSE002	Rahul Mehra	CSE
2	Meridian Analyt...	Data Analyst	21CSE006	Arjun Nair	CSE
2	Meridian Analyt...	Data Analyst	21CSE001	Ananya Rao	CSE
2	Meridian Analyt...	Data Analyst	21IT005	Kanya Pillai	IT
2	Meridian Analyt...	Data Analyst	21EEE003	Swathi Reddy	EEE
2	Meridian Analyt...	Data Analyst	21IT002	Vikram Singh	IT
2	Meridian Analyt...	Data Analyst	21CSE003	Priya Sharma	CSE

Figure 10.13: Reports Screen

10.14 Database Reflection

To demonstrate the connection between the Java Swing application and MySQL database, a database reflection test was performed. The process shows the database state before the UI operation, the operation performed through the application, the success message, and the updated database record.

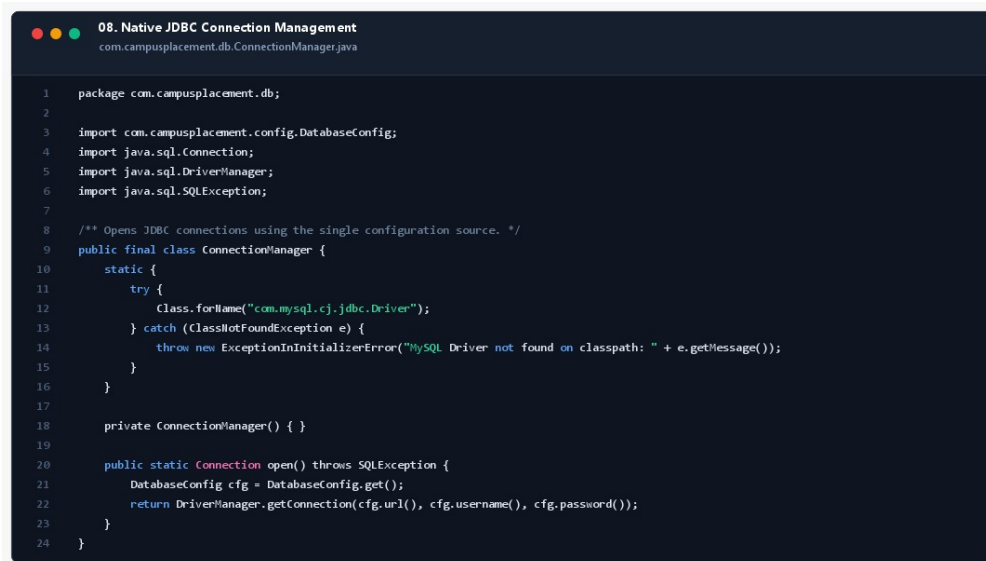
11. IMPLEMENTATION DETAILS

11.1 Technology Stack

The system is developed using Java Swing with MySQL as the database. JDBC is used for database connectivity, Maven for dependency management, and Git/GitHub for version control.

Table 11.1: Technology Stack

Technology	Purpose
Java	Application development
Java Swing	User interface
JDBC	Database connectivity
MySQL	Database
Maven	Dependency management
Git/GitHub	Version control



```

08. Native JDBC Connection Management
com.campusplacement.db.ConnectionManager.java

1  package com.campusplacement.db;
2
3  import com.campusplacement.config.DatabaseConfig;
4  import java.sql.Connection;
5  import java.sql.DriverManager;
6  import java.sql.SQLException;
7
8  /** Opens JDBC connections using the single configuration source. */
9  public final class ConnectionManager {
10     static {
11         try {
12             Class.forName("com.mysql.cj.jdbc.Driver");
13         } catch (ClassNotFoundException e) {
14             throw new ExceptionInInitializerError("MySQL Driver not found on classpath: " + e.getMessage());
15         }
16     }
17
18     private ConnectionManager() { }
19
20     public static Connection open() throws SQLException {
21         DatabaseConfig cfg = DatabaseConfig.get();
22         return DriverManager.getConnection(cfg.url(), cfg.username(), cfg.password());
23     }
24 }

```

Figure 11.1: JDBC Database Connection

11.2 Application Architecture

The application follows a layered architecture:

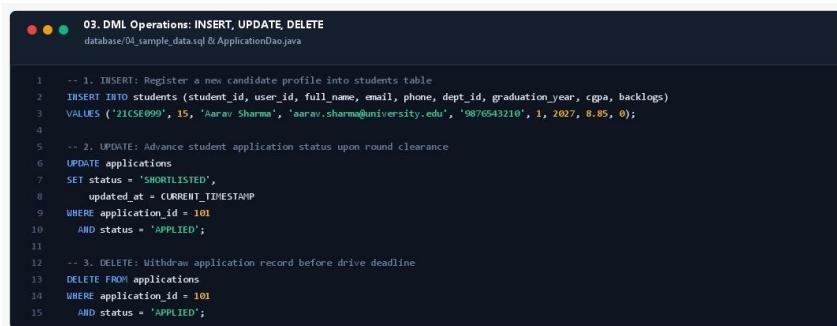
Swing UI → Service Layer → DAO Layer → JDBC → MySQL

The service layer handles business rules, while the DAO layer manages database operations.

11.3 Database and Business Logic

JDBC connects the application to MySQL. DAO classes perform database operations, while service classes handle authentication, eligibility, applications, selection, and offers.

The eligibility process checks CGPA, backlogs, department, and skills before allowing an application.



```
03. DML Operations: INSERT, UPDATE, DELETE
database/04_sample_data.sql @ ApplicationDao.java

1  -- 1. INSERT: Register a new candidate profile into students table
2  INSERT INTO students (student_id, user_id, full_name, email, phone, dept_id, graduation_year, cgpa, backlogs)
3  VALUES ('21CSE099', 15, 'Aarav Sharma', 'aarav.sharma@university.edu', '9876543210', 1, 2027, 8.85, 0);
4
5  -- 2. UPDATE: Advance student application status upon round clearance
6  UPDATE applications
7  SET status = 'SHORTLISTED',
8  updated_at = CURRENT_TIMESTAMP
9  WHERE application_id = 101
10 AND status = 'APPLIED';
11
12 -- 3. DELETE: Withdraw application record before drive deadline
13 DELETE FROM applications
14 WHERE application_id = 101
15 AND status = 'APPLIED';
```

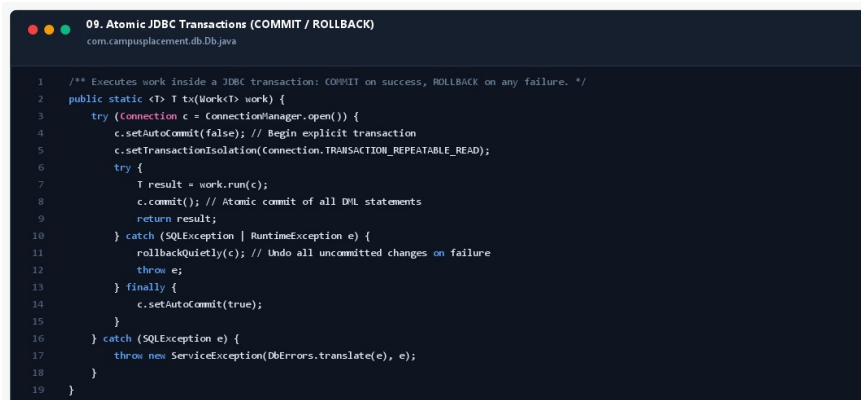
Figure 11.2: DAO and DML Database Operations

11.4 Transactions and Validation

Application submission uses transactions with COMMIT and ROLLBACK to prevent incomplete operations.

```
connection.setAutoCommit(false);
validateEligibility();
insertApplication();
connection.commit();
```

Input validation is also used for CGPA, marks, backlogs, required fields, and duplicate applications.



```
09. Atomic JDBC Transactions (COMMIT / ROLLBACK)
com.campusplacement.db.Db.java

1  /** Executes work inside a JDBC transaction: COMMIT on success, ROLLBACK on any failure. */
2  public static <T> T tx(Work<T> work) {
3      try (Connection c = ConnectionManager.open()) {
4          c.setAutoCommit(false); // Begin explicit transaction
5          c.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
6          try {
7              T result = work.run(c);
8              c.commit(); // Atomic commit of all DML statements
9              return result;
10         } catch (SQLException | RuntimeException e) {
11             rollbackQuietly(c); // Undo all uncommitted changes on failure
12             throw e;
13         } finally {
14             c.setAutoCommit(true);
15         }
16     } catch (SQLException e) {
17         throw new ServiceException(DbErrors.translate(e), e);
18     }
19 }
```

Figure 11.3: JDBC Transaction with COMMIT and ROLLBACK

11.5 Authentication and Reports

The system uses role-based access for Officer and Student users. Authentication is handled through the login, service, DAO, and database layers. Reporting is implemented using ReportService, ReportDao, and CsvExporter

12. TESTING

Testing was performed to verify both normal operations and invalid scenarios. The testing covers application functionality, database constraints, transactions, and user interface operations.

12.1 Functional Testing

Table 12.1: Functional Testing – Test Cases

Test Case	Expected Result
Valid officer login	Officer dashboard opens
Valid student login	Student dashboard opens
Invalid login	Login rejected
Add student	Student inserted
Duplicate student ID	Operation rejected
Duplicate email	Operation rejected
Invalid CGPA	Validation error
Negative backlog	Validation error
Create company	Company created
Create job profile	Profile created
Create placement drive	Drive created
Configure eligibility	Criteria saved
Eligible student applies	Application accepted
Ineligible student applies	Application rejected

Table 12.1: Functional Testing – Test Cases (continued)

Duplicate application	Application rejected
Closed drive application	Application rejected
Expired deadline	Application rejected
Issue offer	Offer created
Duplicate offer	Rejected
Accept/Reject offer	Status updated

These test cases cover the main student and officer workflows.

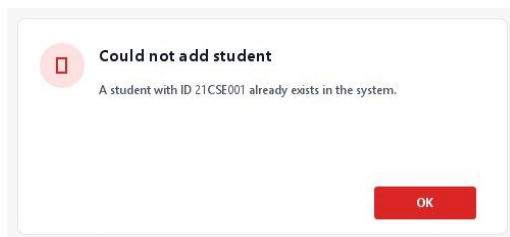


Figure 12.1: Duplicate Student ID Test

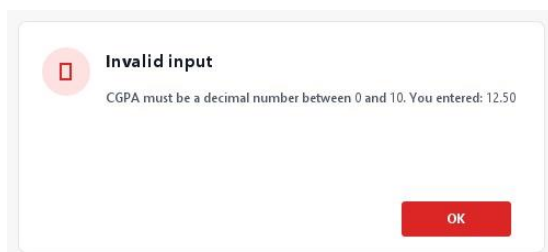


Figure 12.2: Invalid CGPA Validation Test

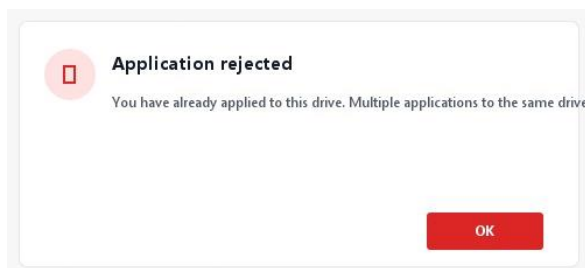


Figure 12.3: Duplicate Application Test

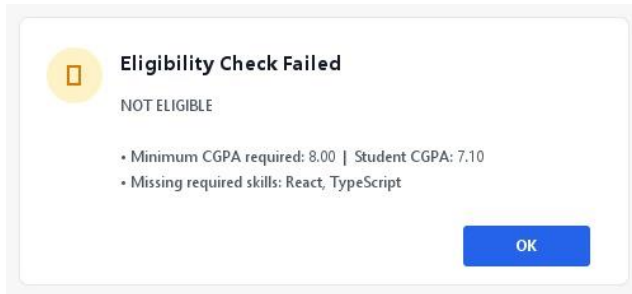


Figure 12.4: Eligibility Validation Test

12.2 Database Testing

The database was tested for:

- Primary key constraints
- Foreign key constraints
- Unique constraints
- Check constraints
- Duplicate application prevention

12.3 Transaction Testing

Transactions were tested to ensure that successful operations are committed and failed operations are rolled back.

Successful operation → COMMIT

Failed operation → ROLLBACK

This prevents incomplete records from being stored in the database.

12.4 UI Testing

The interface was tested for:

- Login and navigation
- Forms and CRUD operations
- Error messages
- Student workflow
- Officer workflow
- Reports
- Database synchronization

12.5 Test Results

Table 12.2: Test Results

Test ID	Test Description	Expected	Actual	Status
TC-01	Valid officer login	Dashboard opens	Dashboard opened	PASS
TC-02	Invalid login	Error shown	Error shown	PASS
TC-03	Duplicate application	Rejected	Rejected	PASS
TC-04	Ineligible student	Application blocked	Application blocked	PASS
TC-05	Valid offer acceptance	Status updated	Status updated	PASS

13. CONCLUSION AND FUTURE ENHANCEMENTS

13.1 Conclusion

The Campus Placement and Recruitment Drive Management System successfully provides a centralized system for managing student information, companies, job profiles, placement drives, eligibility, applications, selection rounds, and offers.

The use of Java Swing, JDBC, and MySQL provides a structured desktop-based solution with proper database relationships, constraints, validation, and transaction handling. The normalized database design also helps maintain data consistency and reduce redundancy.

13.2 Future Enhancements

The system can be enhanced in the future by adding:

- Web-based access
- Mobile application
- Email/SMS notifications
- Resume upload and management
- Advanced placement analytics
- Automated eligibility recommendations
- Integration with external recruitment platforms
- Cloud deployment

14. REFERENCES

1. Oracle. *Java Platform, Standard Edition 17 Documentation*.
<https://docs.oracle.com/en/java/javase/17/>
2. Oracle. *Java SE 17 API Specification*.
<https://docs.oracle.com/en/java/javase/17/docs/api/index.html>
3. Oracle. *JDBC Basics – JDBC Database Access*.
<https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>
4. Oracle. *JDBC Overview and Database Connectivity*.
<https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/overview-of-JDBC.html>
5. Oracle. *MySQL 8.4 Reference Manual*.
<https://dev.mysql.com/doc/refman/8.4/en/>
6. Apache Software Foundation. *Apache Maven Documentation*.
<https://maven.apache.org/>
7. Apache Software Foundation. *Maven Guides – Build Fundamentals and Dependency Management*.
<https://maven.apache.org/guides/>
8. GitHub. *GitHub Documentation*.
<https://docs.github.com/en>
9. GitHub. *Using GitHub – Getting Started and Collaboration*.
<https://docs.github.com/en/get-started/using-github>

15. Contribution of Each Member

Table 15.1: Contribution of Each Member

Name	Contribution
Sahasra Devunuri	Database design, ER diagram, normalization, SQL queries and database implementation
Siri Raaga	JDBC connectivity and basic database integration
Riddhi	UI design, Swing interfaces, forms, navigation and reports
Mohak Gupta	Testing, validation, screenshots, documentation and presentation preparation

16. GitHub Repository

The project source code, database scripts, and related project files can be maintained in a GitHub repository. GitHub provides version control and makes it easier for team members to collaborate, manage changes, and maintain the project documentation.

GitHub Repository: <https://github.com/RiddhiAgarwal2025/campus-placement-management-system>

Repository contents:

- Java source code
- MySQL database scripts
- Maven configuration
- Project documentation
- README file
- Screenshots and supporting files